

```

"""
Data Cleaning & Reporting Automation
-----
Task: Automate data cleaning and reporting workflows.

Pipeline:
1. Load raw (messy) dataset
2. Detect & report data quality issues
3. Clean: handle missing values, duplicates, inconsistent formatting
4. Generate an automated HTML report with visual summaries
"""

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from datetime import datetime

RNG = np.random.default_rng(7)
OUT_DIR = "/home/claude/project2"

# -----
# 1. LOAD / GENERATE RAW (MESSY) DATASET
# -----
# (Swap this block with pd.read_csv("your_file.csv") for real data.)

n = 120
names = [f"Customer_{i}" for i in range(1, n + 1)]
cities_clean = ["Chennai", "Mumbai", "Delhi", "Bangalore", "Hyderabad"]
# messy variants of the same cities (case/whitespace inconsistencies)
city_variants = ["chennai", "CHENNAI", " Chennai", "mumbai", "Mumbai ",
                 "delhi", "Delhi", "bangalore", "Bengaluru", "hyderabad", "Hyder

raw = pd.DataFrame({
    "customer_id": range(1, n + 1),
    "name": names,
    "city": RNG.choice(city_variants, n),
    "signup_date": pd.date_range("2024-01-01", periods=n, freq="3D").astype(str),
    "purchase_amount": RNG.normal(2500, 800, n).round(2),
    "email": [f"user{i}@mail.com" for i in range(1, n + 1)],
})

# Inject realistic messiness

```

```

raw.loc[RNG.choice(n, 12, replace=False), "purchase_amount"] = np.nan #
raw.loc[RNG.choice(n, 8, replace=False), "city"] = None #
raw = pd.concat([raw, raw.iloc[[3, 10, 22, 50]]], ignore_index=True) #
raw.loc[RNG.choice(len(raw), 5, replace=False), "purchase_amount"] = -999 #
raw.loc[15, "email"] = "not-an-email" #

raw.to_csv(f"{OUT_DIR}/raw_data.csv", index=False)

# -----
# 2. DATA QUALITY REPORT (before cleaning)
# -----
issues_before = {
    "Total rows": len(raw),
    "Duplicate rows": raw.duplicated().sum(),
    "Missing purchase_amount": raw["purchase_amount"].isna().sum(),
    "Missing city": raw["city"].isna().sum(),
    "Invalid purchase_amount (negative)": (raw["purchase_amount"] < 0).sum(),
    "Invalid email format": (~raw["email"].str.contains(r"^[\\w\\.-]+@[\\w\\.-]+\\.\\w+"))
    "Inconsistent city spellings": raw["city"].dropna().str.strip().str.lower().n
}

# -----
# 3. AUTOMATED CLEANING
# -----
df = raw.copy()

# 3a. Remove exact duplicate rows
df = df.drop_duplicates()

# 3b. Fix invalid / negative purchase amounts -> treat as missing, then impute
df.loc[df["purchase_amount"] < 0, "purchase_amount"] = np.nan
df["purchase_amount"] = df["purchase_amount"].fillna(df["purchase_amount"].median)
df["purchase_amount"] = df["purchase_amount"].round(2)

# 3c. Standardize city names (strip whitespace, title case, fix known aliases)
city_map = {"bengaluru": "Bangalore"}
df["city"] = df["city"].astype(str).str.strip().str.lower()
df["city"] = df["city"].replace(city_map)
df["city"] = df["city"].str.title()
df.loc[df["city"] == "None", "city"] = df["city"].mode()[0] # impute missing cit

# 3d. Fix invalid emails -> flag them
df["email_valid"] = df["email"].str.contains(r"^[\\w\\.-]+@[\\w\\.-]+\\.\\w+$", na=False)

# 3e. Parse dates properly
df["signup_date"] = pd.to_datetime(df["signup_date"], errors="coerce")

```

```

df = df.reset_index(drop=True)
df.to_csv(f"{OUT_DIR}/cleaned_data.csv", index=False)

# -----
# 4. DATA QUALITY REPORT (after cleaning)
# -----
issues_after = {
    "Total rows": len(df),
    "Duplicate rows": df.duplicated().sum(),
    "Missing purchase_amount": df["purchase_amount"].isna().sum(),
    "Missing city": (df["city"] == "None").sum(),
    "Invalid purchase_amount (negative)": (df["purchase_amount"] < 0).sum(),
    "Invalid email format": (~df["email_valid"]).sum(),
    "Inconsistent city spellings": df["city"].nunique(),
}

print("=== BEFORE CLEANING ===")
for k, v in issues_before.items():
    print(f"{k:38s}: {v}")
print("\n=== AFTER CLEANING ===")
for k, v in issues_after.items():
    print(f"{k:38s}: {v}")

# -----
# 5. VISUAL SUMMARIES
# -----
fig, axes = plt.subplots(2, 2, figsize=(12, 9))

# Issue counts before vs after
labels = list(issues_before.keys())[1:6]
before_vals = [issues_before[k] for k in labels]
after_vals = [issues_after[k] for k in labels]
x = np.arange(len(labels))
axes[0, 0].bar(x - 0.2, before_vals, width=0.4, label="Before", color="#ef4444")
axes[0, 0].bar(x + 0.2, after_vals, width=0.4, label="After", color="#16a34a")
axes[0, 0].set_xticks(x)
axes[0, 0].set_xticklabels(labels, rotation=30, ha="right", fontsize=8)
axes[0, 0].set_title("Data Quality Issues: Before vs After")
axes[0, 0].legend()

# City distribution after cleaning
df["city"].value_counts().plot(kind="bar", ax=axes[0, 1], color="#2563eb")
axes[0, 1].set_title("Customer Count by City (cleaned)")
axes[0, 1].set_xlabel("")

# Purchase amount distribution
axes[1, 0].hist(df["purchase_amount"], bins=20, color="#f59e0b", edgecolor="white")

```

```

axes[1, 0].set_title("Purchase Amount Distribution (cleaned)")
axes[1, 0].set_xlabel("Purchase Amount")

# Signups over time
df.set_index("signup_date").resample("W").size().plot(ax=axes[1, 1], color="#8b5c
axes[1, 1].set_title("Weekly Signups (cleaned)")
axes[1, 1].set_ylabel("Signups")

plt.tight_layout()
plt.savefig(f"{OUT_DIR}/report_charts.png", dpi=150)

# -----
# 6. AUTOMATED HTML REPORT
# -----

html = f"""
<html><head><meta charset="utf-8">
<title>Data Cleaning & Reporting Automation</title>
<style>
body {{ font-family: Arial, sans-serif; margin: 40px; color: #1e293b; }}
h1 {{ color: #1d4ed8; }}
h2 {{ color: #0f766e; margin-top: 32px; }}
table {{ border-collapse: collapse; width: 100%; margin: 12px 0; }}
th, td {{ border: 1px solid #e2e8f0; padding: 8px 12px; text-align: left; }}
th {{ background: #f1f5f9; }}
.tag {{ display:inline-block; padding:2px 10px; border-radius:12px; font-size:12p
.ok {{ background:#dcfce7; color:#166534; }}
img {{ max-width: 100%; border: 1px solid #e2e8f0; border-radius: 8px; margin-top
</style></head><body>

<h1>Data Cleaning & Reporting Automation</h1>
<p>Generated automatically on {datetime.now().strftime('%d %b %Y, %H:%M')}</p>

<h2>1. Data Quality: Before Cleaning</h2>
<table><tr><th>Check</th><th>Count</th></tr>
{''.join(f"<tr><td>{k}</td><td>{v}</td></tr>" for k, v in issues_before.items())}
</table>

<h2>2. Data Quality: After Cleaning</h2>
<table><tr><th>Check</th><th>Count</th></tr>
{''.join(f"<tr><td>{k}</td><td>{v}</td></tr>" for k, v in issues_after.items())}
</table>
<p><span class="tag ok">Cleaning complete</span></p>

<h2>3. Cleaning Steps Applied</h2>
<ul>
<li>Removed {issues_before['Duplicate rows']} duplicate rows</li>
<li>Imputed missing/invalid purchase amounts with median value</li>

```

```
<li>Standardized inconsistent city names (case, whitespace, aliases like "Bengalu
<li>Flagged invalid email formats for follow-up</li>
<li>Parsed signup dates into proper datetime format</li>
</ul>
```

```
<h2>4. Visual Summary</h2>
```

```

```

```
</body></html>
```

```
"""
```

```
with open(f"{OUT_DIR}/automated_report.html", "w") as f:
    f.write(html)
```

```
print(f"\nReport saved to {OUT_DIR}/automated_report.html")
```